NLU course project

Seyed Morteza Mojtabavi (256328)

University of Trento

seyed.mojtabavi@studenti.unitn.it

The following is the report for the second part of the project: Natural Language Understanding.

1. Introduction

This part of the project covers two ATIS tasks trained jointly: slot filling, a sequence-labeling problem evaluated with CoNLL F1, and intent classification, evaluated with accuracy. Part A starts from a unidirectional LSTM joint model (`ModelIAS`) and incrementally adds bidirectionality and dropout, followed by a two-round sequential hyperparameter search. Part B instead fine-tunes a pre-trained BERT-base-uncased encoder for the same two tasks, following Chen et al. (2019), reusing as much of Part A's training/evaluation infrastructure as the different backbone allows. Both parts rank experiments by the average of Slot F1 and Intent Accuracy, alongside the two required metrics individually.

2. Implementation details

2.1. Part A

I implemented `ModelIAS`, a joint intent/slot LSTM, as the Baseline. While isolating the two required modifications, I found the codebase's Baseline and Bidirectional configurations were silently training with dropout active by a hidden default (`dropout_rate=0.1`), since the model always instantiates a dropout layer regardless of configuration. To make "no dropout" [1] an explicit, verifiable choice rather than an accidental side effect, I retrained both with `dropout_rate=0.0`, mathematically equivalent to omitting the layer entirely. For bidirectionality, enabling the LSTM's `bidirectional` flag [2] required concatenating the forward/backward final hidden states before the intent head. I also traced and fixed two further issues while validating the pipeline: the label vocabulary was built by iterating a raw Python `set()`, whose order is randomized per process and silently scrambles any checkpoint reloaded later; and the sequential tuner's `batch_size` trials were all training with one fixed-size data loader instead of a per-trial one. The two-round hyperparameter search ranks trials by the average of Slot F1 and Intent Accuracy rather than F1 alone, since the task is explicitly joint.

2.2. Part B

Part B fine-tunes BERT-base-uncased [3, 4] for the same joint task, reusing Part A's training loop structure where the different backbone allows. Following Chen et al. [5], sub-word tokenization is handled by assigning each word's slot label only to its first WordPiece sub-token, with an ignore index on every other position, so cross-entropy and evaluation are both driven only by first-sub-token positions; the `[CLS]` token's pooled output feeds the intent head directly, requiring no length-based sequence sorting since attention, unlike the LSTM, is not order-dependent. For the baseline, I matched the paper's reported `lr=5e-5`, `batch_size=128`, and `dropout=0.1`, but deliberately kept AdamW rather than switching to the paper's plain Adam, then ran the same averaged-metric sequential search used in Part A.

3. Results

3.1. Part A

Bidirectionality gave the largest single gain on its own, letting the slot head use both left and right context per token and giving the intent head a fuller summary of the whole utterance. Dropout added a further improvement once properly isolated from the earlier hidden-default issue. The two-round sequential search then settled on `lr=0.001`, `hidden_size=200`, `emb_size=200`, `dropout_rate=0.7`, `batch_size=64`, and `clip=0.25`, reaching a final Test F1 of 0.9513 and Intent Accuracy of 0.9653.

Table 1: *Best results for each configuration (Part A).*

Configuration	Test F1	Test Intent Acc
Baseline (unidirectional, no dropout)	0.9186	0.9339
+ Bidirectional	0.9375	0.9362
+ Dropout	0.9431	0.9518
Final (tuned)	**0.9513**	**0.9653**

3.2. Part B

The BERT baseline, matching the paper's reported learning rate, batch size, and dropout (but keeping AdamW instead of the paper's plain Adam), already lands close to its published ATIS numbers. The subsequent search found a different optimum for this codebase and split: a smaller learning rate, a much smaller batch size, and a nonzero weight decay the paper never used, together reaching a final Test F1 of 0.9599 and Intent Accuracy of 0.9742 — reproduced exactly when retraining from scratch with the tuned configuration.

Table 2: *Best results for each configuration (Part B).*

Configuration	Test F1	Test Intent Acc
Baseline (AdamW)	0.9569	0.9742
Final (tuned)	**0.9599**	**0.9742**

4. References

[1] PyTorch, "Dropout," https://docs.pytorch.org/docs/stable/generated/torch.nn.Dropou

[2] ——, "Long short-term memory (lstm)," https://docs.pytorch.org/docs/stable/generated/torch.nn.LSTM.html.

[3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2019.

[4] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 38–45.

[5] Q. Chen, Z. Zhuo, and W. Wang, "Bert for joint intent classification and slot filling," *arXiv preprint arXiv:1902.10909*, 2019.